

Análise Comparativa de Compressão: LZW vs. Huffman em Cenários de Comunicação Tática (IoMT)

Carlos E. F. Caldas, Gabriela J. Perez, João G. C. de Andrade, Júlia M. Carvalho, Leonardo P. Estigarriba, Lucas F. Ataíde, Luís E. G. Mendes, Nicholas F. Garcia¹

¹Universidade Federal do ABC (UFABC)
Santo André – SP – Brasil

{carlos.finatti, gabriela.perez, caetano.andrade}@aluno.ufabc.edu.br

{julia.muniz, leonardo.pavlavicius}@aluno.ufabc.edu.br

{lucas.ataide, luis.emilio, fernandez.garcia}@aluno.ufabc.edu.br

Abstract. *Tactical military communication requires minimizing radio transmission time and bandwidth usage to preserve battery and reduce interception by the enemy. This paper compares the lossless compression algorithms Huffman and Lempel-Ziv-Welch (LZW), applied to datasets representing typical tactical messages, analyzing them in terms of complexity, practical execution time and compression ratio, in addition to comparing the ease of implementation of each algorithm. Results show that LZW compresses more efficiently, especially in redundant files, but it takes more processing time; Huffman compresses less but faster. Algorithm choice should therefore follow the scenario's operational priority: LZW when minimizing airtime is critical, Huffman when processing power and battery are limited.*

Keywords: Data Compression, LZW, Huffman Encoding, Tactical Communication, IoMT.

Resumo. *A comunicação militar tática exige minimizar o tempo de transmissão via rádio e o uso de banda, preservando bateria e reduzindo a interceptação pelo inimigo. Este trabalho compara os algoritmos de compressão sem perdas Huffman e Lempel-Ziv-Welch (LZW), aplicados a datasets que representam mensagens táticas típicas, analisando-os quanto à complexidade, ao tempo prático de execução e à taxa de compressão, além de comparar a facilidade de implementação de cada algoritmo. Os resultados mostram que o LZW é mais eficiente na redução do tamanho dos dados, especialmente em arquivos redundantes, mas exige maior tempo de processamento; o Huffman apresenta menor taxa de compressão, porém com execução mais rápida. Conclui-se que a escolha deve seguir a prioridade do cenário: LZW quando minimizar o airtime é crítico; Huffman quando processamento e bateria são limitados.*

Palavras-chave: Compressão de Dados, LZW, Codificação de Huffman, Comunicação Tática, IoMT.

1. Introdução

Uma das principais armas na guerra é a informação. Ao transmitir as informações, para maior eficiência de aparelhos como o rádio e menores chances de detecção, são utilizadas técnicas de compressão de dados sem perdas. A ideia de representar informação de maneira compacta não é recente: já no Código Morse (1838), os símbolos mais curtos eram atribuídos às letras mais frequentes, o mesmo princípio guloso que seria formalizado matematicamente por Huffman [Huffman 1952, Cover and Thomas 2006]. A abordagem baseada em dicionário, utilizada por meio do LZW, tem origem no trabalho de Terry Welch [Welch 1984], otimização do algoritmo LZ78 [Ziv and Lempel 1978] que se tornaria a base de formatos como o GIF.

Neste artigo, objetiva-se analisar comparativamente essas duas técnicas de compressão, o algoritmo de Huffman e de Lempel-Ziv-Welch (LZW), e concluir sobre qual é mais eficiente em termos de complexidade de tempo, complexidade de espaço, tempo prático de execução das operações e facilidade de implementação. Será feita a implementação dos algoritmos e testes em *datasets* simulando os tipos de mensagem mais utilizados pelos militares. Essa escolha justifica-se pelo fato de que cenários de comunicação tática impõem restrições simultâneas de banda, energia e sigilo que raramente são avaliadas em conjunto na literatura [Krokosz et al. 2023, Dias et al. 2026], motivando a comparação proposta no presente artigo. Como contribuição deste trabalho, destaca-se a aplicação prática desses critérios sobre um conjunto de dados que representa cenários táticos variados, o que preenche uma lacuna deixada pelos trabalhos discutidos na Revisão Bibliográfica.

O restante do trabalho está organizado da seguinte forma: Explicação do Problema de Pesquisa, abordará o contexto e a motivação para a realização do trabalho; Revisão Bibliográfica, na qual são verificados outros artigos relacionados a esse tópico para analisar o que já foi feito e explicar o diferencial desse estudo; Explicação da Aplicação Computacional, seção que descreverá os algoritmos e estrutura de dados utilizados, além das complexidades assintóticas de cada um; Análise dos Resultados, apresenta as análises comparativas entre os dois algoritmos de acordo com os testes realizados e resultados esperados; Conclusões Gerais e Trabalhos Futuros, última seção, que recapitulará o objetivo do projeto, apresentará as conclusões obtidas e tratará sobre trabalhos futuros.

2. Explicação do Problema de Pesquisa

O problema de pesquisa abordado refere-se a um estudo de caso realístico que simula a troca de informações durante um conflito militar no contexto da IoMT, situação em que os recursos são limitados e devem ser preservados ao máximo. Por essa razão, é de extrema importância economizar recursos, como bateria dos rádios, e manter a discrição. Durante uma transmissão de rádio, há uma janela de vulnerabilidade na qual os operadores podem ser detectados, através da interceptação do sinal emitido. Portanto, quanto menor o tempo que o rádio fica ativo (*airtime*), menores são as chances de detecção pelos inimigos (Low Probability of Intercept/Detection – LPI/LPD).

Para isso, são utilizados métodos de compressão como LZW e Huffman, os quais permitem reduzir o tamanho das mensagens para que sejam transmitidas de maneira compacta e rápida, garantindo a comunicação de informações como coordenadas geográficas, dados de telemetria e relatórios de inteligência. A escolha desses tipos de mensagem

busca representar a diversidade de tráfego esperada em um cenário de IoMT: desde comunicação operacional direta (mensagens e coordenadas) até dados de sensoriamento de campo (telemetria e biometria) e material de reconhecimento (mapas de satélite), permitindo avaliar o comportamento dos algoritmos de compressão sobre diferentes padrões de redundância e estrutura de dados.

3. Revisão Bibliográfica

No trabalho de [Ketshabetswe et al. 2021], realizou-se uma revisão ampla e comparativa de algoritmos de compressão para Redes de Sensores Sem Fio (*Wireless Sensor Networks* – WSNs). Os autores demonstraram que a eficiência do método ideal depende essencialmente do padrão de dados trafegado, comprovando que abordagens estatísticas, como a codificação de Huffman [Cormen et al. 2009], possuem melhor rendimento quando a frequência dos símbolos individuais é bem definida e pode ser calculada previamente, enquanto abordagens baseadas em dicionário apresentam desempenho superior na presença de sequências, de caracteres repetidos. Além disso, a pesquisa de [Nassra and Capella 2023] focou no uso dessas técnicas aplicadas a redes de sensores biomédicos (*Wireless Body Sensor Networks*) habilitadas para IoT (*Internet of Things*). Em seu estudo, os autores destacaram que o consumo de energia provocado pela transmissão via rádio supera em muito o custo computacional de processamento interno, confirmando a premissa de que comprimir dados localmente é uma estratégia direta para preservar a autonomia de bateria dos dispositivos.

A mudança desses conceitos para o campo de defesa e segurança tática foi analisada por [Krokosz et al. 2023], que explorou a aplicação de algoritmos como Huffman e LZW no escopo de mensagens curtas para a Internet das Coisas Militar (*Internet of Military Things* – IoMT). Os autores pontuaram que, de maneira diferente das redes comerciais, a IoMT enfrenta grandes desafios estratégicos, como a necessidade de manter baixa probabilidade de interceptação de sinal por forças adversárias e a otimização de cargas úteis reduzidas. No entanto, os autores não avaliaram comparativamente o tempo prático de execução desses algoritmos sobre diferentes tipos de mensagens táticas. No contexto de transmissões de rádio em ambientes operacionais restritos, o trabalho de [Dias et al. 2026] contribui ao avaliar experimentalmente o impacto da compressão no consumo de energia e no tempo de ocupação do canal, reforçando a importância de se mitigar o tempo de antena ativa (*airtime*) para reduzir a janela de oportunidade para ataques de guerra eletrônica e interceptação.

Os trabalhos de [Ketshabetswe et al. 2021] e [Nassra and Capella 2023] validam a eficácia de Huffman e LZW em redes de sensores, enquanto os estudos de [Krokosz et al. 2023] e [Dias et al. 2026] descrevem os requisitos e o consumo energético de redes de baixa potência em cenários táticos. Observa-se, contudo, uma lacuna na comparação empírica do desempenho desses algoritmos aplicadas especificamente a mensagens táticas estruturadas e variadas. Desse modo, o presente artigo avalia como o comportamento assintótico e o tempo prático de execução de cada algoritmo respondem à semântica e à estrutura de dados específicas de cenários de conflito.

4. Explicação da Aplicação Computacional

Nesta seção, explica-se a aplicação computacional desenvolvida para comparar os algoritmos de compressão sem perdas Huffman e LZW. Ambos foram implementados em linguagem C++. As subseções a seguir detalham, separadamente os algoritmos, estruturas de dados e análises de complexidade de tempo e espaço de cada técnica.

4.1. Algoritmo Lempel-Ziv-Welch (LZW)

O algoritmo de LZW é implementado por meio da estrutura de dados, tabela *hash* com 4096 posições (12 bits), na qual as primeiras 256 posições são preenchidas com os caracteres da tabela ASCII, em que o código de identificação representa a posição ocupada. Após isso, o vetor da mensagem é percorrido, e a cada nova combinação de caracteres ainda não existentes na tabela, o novo símbolo formado é colocado na próxima posição livre. Portanto, mensagens que possuem redundâncias serão comprimidas de maneira mais eficiente, pois serão realizadas menos operações de inserção na tabela.

Suas complexidades de tempo e espaço são, respectivamente, $O(N)$ e $O(M)$, onde N representa o tamanho da mensagem e M representa o tamanho do dicionário. As operações de inserção e busca na tabela *hash* levam em tempo médio $O(1)$ [Sedgewick and Wayne 2011]. Contudo, nesta aplicação foi estabelecido um limite do dicionário para 4096 posições (12 bits), o que torna a complexidade de espaço auxiliar $O(1)$ na prática. O motivo para essa limitação deve-se ao hardware altamente limitado utilizado pelos militares. Esse mesmo limite de 12 bits é adotado historicamente por implementações do LZW, como o formato GIF [CompuServe Incorporated 1990] (*Graphics Interchange Format*), mostrando que se trata de um parâmetro validado, e não de uma escolha arbitrária. Quando o dicionário atinge sua capacidade máxima, o algoritmo deixa de inserir novas combinações, mas continua comprimindo a mensagem utilizando apenas as entradas já existentes na tabela. Optou-se por não reiniciar o dicionário (*reset*) ao atingir o limite, uma vez que mensagens táticas tendem a ser de curta duração, o que reduz a probabilidade de preenchimento completo da tabela durante uma transmissão.

Antes de apresentar o Algoritmo 1, usaremos ϵ para denotar a palavra vazia, $\cdot$ para a relação de concatenação, e dado um alfabeto Σ , Σ^* denota o conjunto de todas as possíveis palavras formadas com os elementos de Σ .

A descompressão do LZW opera de maneira inversa, reconstruindo o dicionário à medida que os códigos são lidos, sem que seja necessário transmitir a tabela junto com a mensagem. O decodificador mantém sua própria tabela T^{-1} , inicializada com os 256 caracteres ASCII, e ao ler cada código o_i da lista O , recupera a sequência correspondente e a concatena à saída, adicionando uma nova entrada a T^{-1} seguindo a mesma regra usada na compressão. Assim, compressor e descompressor chegam sempre ao mesmo dicionário sem trocarem essa informação explicitamente, característica que torna o LZW atrativo para comunicação tática de baixa banda, já que apenas os códigos precisam ser transmitidos.

Algoritmo 1 LZW (Lempel-Ziv-Welch)

Entrada: (S, M) onde $S \in \Sigma^*$ é uma palavra sobre um alfabeto Σ e $M \in \mathbb{N}$ o tamanho máximo do dicionário, com $|S| =: n$ e $|\Sigma| = 256$.

Saída: $O \in \Gamma^*$, a palavra comprimida de S , onde $\Gamma := \{0, 1, \dots, M-1\}$ é o alfabeto de códigos

1: Seja $T : \Sigma^* \rightarrow \mathbb{N}_0 \mid T(a) = a, \forall a \in \Sigma$

2: $code \leftarrow |\Sigma|$

3: $p \leftarrow S[0]$

4: $O \leftarrow \epsilon$

5: **para** $i \leftarrow 0$ **até** $n - 1$ **faça**

6: **se** $i = n - 1$ **então**

7: $c \leftarrow \epsilon$

8: **senão**

9: $c \leftarrow S[i + 1]$

10: **se** $p \cdot c \in \text{dom}(T)$ **então**

11: $p \leftarrow p \cdot c$

12: **senão**

13: $O \leftarrow O \cdot T(p)$

14: **se** $code < M$ **então**

15: $T(p \cdot c) \leftarrow code$

16: $code \leftarrow code + 1$

17: $p \leftarrow c$

18: $O \leftarrow O \cdot T(p)$

19: **retorne** O

4.2. Algoritmo de Huffman

O algoritmo de Huffman é um método de compressão de dados sem perdas que reduz o tamanho de arquivos ao analisar a frequência de cada caractere presente. Em vez de usar um tamanho fixo de bits para todos os símbolos, ele atribui códigos binários mais curtos para os caracteres que aparecem com maior frequência e códigos mais longos para aqueles que são mais raros, economizando espaço no arquivo final.

Para gerar o código de cada símbolo, o algoritmo guarda um vetor com as respectivas frequências de cada símbolo e cria uma estrutura em formato de árvore binária, construída de baixo para cima na versão iterativa ou de cima para baixo na versão recursiva. Inicialmente, ele trata cada caractere como uma sub-árvore trivial (folha). Então, ele segue, de forma gulosa, unindo as duas sub-árvores de menor frequência e criando um novo símbolo intermediário, o qual será a raiz desta nova sub-árvore. A frequência desta sub-árvore maior será a soma das frequências de seus filhos. Esse processo continua até que todos os símbolos do alfabeto dado estejam incluídos.

Depois que a árvore está inteiramente montada, o algoritmo desce da raiz até todas as folhas. A raiz recebe o prefixo vazio e toda sub-árvore à esquerda recebe o código formado pelo prefixo atual mais o dígito zero, e toda sub-árvore à direita recebe o código formado pelo prefixo atual mais o dígito um. Assim, todas as folhas terão seu código equivalente à sequência de 0's e 1's da raiz até a si mesma. Como os símbolos estão sempre nas folhas da árvore, eles não possuem relação de ancestralidade entre si, assim o código de uma letra nunca será o prefixo do código de outra. Este é o motivo pelo qual este modelo produz uma codificação livre de prefixos.

Como o algoritmo precisa de que sempre tenhamos os dois símbolos de menor frequência é comum a utilização de um *min-heap* para a rápida consulta/remoção deste mínimo e adição de novos símbolos intermediários. Usualmente em bibliotecas padrão de linguagens de programação (como a STL do C++) são encontradas implementações diretas desse *min-heap* (*priority queue*). O Algoritmo 2 apresenta a implementação em sua versão recursiva:

Algoritmo 2 Huffman

Entrada: (Σ, f) um alfabeto e uma função de frequências

Saída: σ uma codificação binária livre de prefixos ótima

```

1: se  $|\Sigma| = 1$  então
2:   Seja  $u \in \Sigma$ 
3:   retorne  $\{(u, 0)\}$ 
4: se  $|\Sigma| = 2$  então
5:   Seja  $u, v \in \Sigma$ 
6:   retorne  $\{(u, 0), (v, 1)\}$ 
7: Seja  $u \in \Sigma$  tal que  $f(u) = \min(\{f(a) \mid a \in \Sigma\})$ 
8: Seja  $v \in \Sigma \setminus \{u\}$  tal que  $f(v) = \min(\{f(a) \mid a \in \Sigma\})$ 
9: Tome um símbolo  $\alpha \notin \Sigma$ 
10:  $\Sigma' := \Sigma \setminus \{u, v\} \cup \{\alpha\}$ 
11:  $f' := \begin{cases} f'(a) = f(u) + f(v) & \text{se } \alpha = a \\ f'(a) = f(a) & \text{cc} \end{cases}$ 
12:  $\sigma' \leftarrow \text{Huffman}(\Sigma', f')$ 
13: retorne  $\sigma := \sigma' \cup \{(u, 0), (v, 1)\}$ 

```

Note que, para uma versão iterativa do algoritmo bastava guardar Σ como uma *priority queue* e f com um vetor. E enquanto a fila não estivesse vazia continuaríamos a retirar os dois elementos de menor frequência e adicionar um novo com a frequência igual a soma dos dois removidos. Além disso, nossa codificação binária livre de prefixos ótima seria a árvore binária, com os elementos criados na recursão (α) sendo os nós internos, enquanto as folhas sendo os elementos de nosso alfabeto (Σ).

Para a análise de complexidade temporal do algoritmo de Huffman, consideram-se dois casos. Primeiro, a montagem da codificação binária livre de prefixos ótima: esta é dominada pelas remoções de símbolos com menor frequência, sabemos que um *min-heap* consegue buscar um menor elemento em $O(1)$ e removê-lo em tempo $O(\lg n)$. Como precisaremos fazer essa remoção ao menos $O(n)$ vezes, onde $n := |\Sigma|$, temos que o algoritmo demora tempo proporcional a $O(n \lg n)$ para montar a codificação. Entretanto, precisaremos passar por cada caractere da entrada e substituí-lo por sua codificação já calculada, assim essa varredura custaria $O(|S|)$ onde S é nossa entrada. Como $n \in O(|S|)$ temos que a complexidade geral de tempo é $O(|S| + n \lg n)$. Já para a análise espacial, temos que o espaço utilizado é proporcional a $O(n)$, visto que, a fila de prioridades vai armazenar no máximo o número de símbolos, além disso, a árvore formada pelo algoritmo vai formar exatamente $n - 1$ nós internos e terá n folhas, somando assim $2n - 1$ nós, o que de fato mostra que o algoritmo de Huffman consome espaço equivalente a $O(n)$, como mencionado acima [Cormen et al. 2009]. A descompressão do Huffman é mais simples que a do LZW: basta enviar o dicionário junto com a codificação. Assim, quando

o receptor for descomprimir o arquivo, ele percorre cada código para identificar o seu caractere original.

5. Análise dos Resultados

A seguir, são comentados os resultados obtidos experimentalmente ao aplicar os algoritmos de Huffman e LZW aos arquivos de teste que representam os cenários de comunicação tática descritos na Explicação do Problema de Pesquisa. A Tabela 1 resume os valores brutos obtidos para os 14 arquivos avaliados. O conjunto de dados inclui cinco arquivos únicos (mensagens, coordenadas, relatório, telemetria e biometria) e três famílias de arquivos de mesmo tipo de conteúdo em três tamanhos distintos (mapas de satélite, *codebooks* e Morse binário, cada um em 6 kB, 251 kB e 3001 kB), o que permite isolar o efeito do tamanho do arquivo em relação ao tipo de conteúdo sobre o desempenho de cada algoritmo.

Tabela 1. Desempenho comparativo entre os algoritmos Huffman e LZW em diferentes conjuntos de dados táticos.

Arquivo	Tam. Orig. (kB)	Comprimido (kB)		T. Compressão (ms)		T. Descompressão (ms)		Taxa Compressão (%)	
		Huffman	LZW	Huffman	LZW	Huffman	LZW	Huffman	LZW
mensagens	1199	759	327	444.28	909.37	52.10	114.51	36.70	72.73
coordenadas	1648	767	396	512.78	1231.11	55.57	138.34	53.46	75.97
relatorio	4645	2699	1072	1676.79	3802.73	206.12	385.27	41.89	76.92
telemetria_misseis	1539	986	388	539.25	1119.00	61.55	139.50	35.93	74.79
biometria_soldado	1474	922	266	515.24	1080.20	57.66	108.02	37.45	81.95
mapa_satelite_1	6	2	1	2.06	4.29	0.20	0.61	66.67	83.33
mapa_satelite_2	251	69	4	77.33	219.98	5.03	9.30	72.51	98.41
mapa_satelite_3	3001	842	21	971.36	2670.45	63.71	89.70	71.94	99.30
codebook_1	6	4	3	4.45	4.40	0.39	1.87	33.33	50.00
codebook_2	251	158	57	94.58	184.44	11.53	22.49	37.85	77.29
codebook_3	3001	1870	657	1068.25	2122.80	132.13	239.53	37.69	78.11
morse_binario_1	6	2	2	1.94	4.32	0.21	1.18	66.67	66.67
morse_binario_2	251	79	20	70.27	202.67	6.16	14.41	68.53	92.03
morse_binario_3	3001	947	218	902.41	2350.48	72.76	137.00	68.44	92.74

Conforme a Figura 1, o algoritmo LZW obteve taxas de compressão superiores ao Huffman em praticamente todos os arquivos testados, com destaque para os arquivos de maior redundância interna, como *mapa_satelite_2* e *mapa_satelite_3*, nos quais o LZW ultrapassou 98% de taxa de compressão, contra cerca de 70% do Huffman nos mesmos arquivos. Já no arquivo *morse_binario_1*, ambos os algoritmos apresentaram desempenho idêntico (66,67%), o que sugere que, quando o alfabeto de entrada é reduzido e a frequência dos símbolos já é naturalmente concentrada, a vantagem do LZW sobre o Huffman diminui.

A Figura 2 mostra que a economia em kilobytes depende do tamanho original do arquivo: arquivos grandes, como *relatorio* (4645 kB), geram reduções absolutas muito maiores do que arquivos pequenos, como *codebook_1* (6 kB), independente da taxa percentual.

Quanto ao tempo de execução, a Figura 3 evidencia que o Huffman é consistentemente mais rápido que o LZW para compressão, e essa diferença se acentua à medida que o tamanho do arquivo aumenta: para o arquivo *relatorio* (4645 kB), o Huffman levou 1676,79 ms contra 3802,73 ms do LZW, ou seja, o LZW consumiu mais que o dobro do tempo. O mesmo padrão se repete no tempo de descompressão (Tabela 1), o que

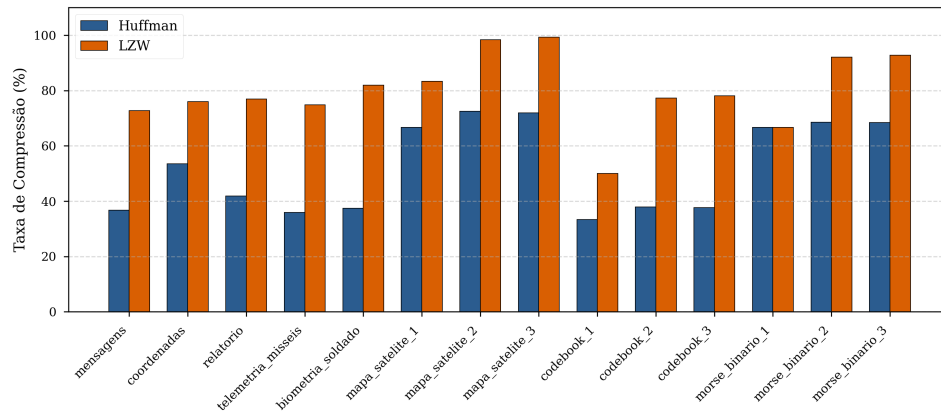


Figura 1. Comparativo da taxa de compressão (%) obtida pelos algoritmos Huffman e LZW.

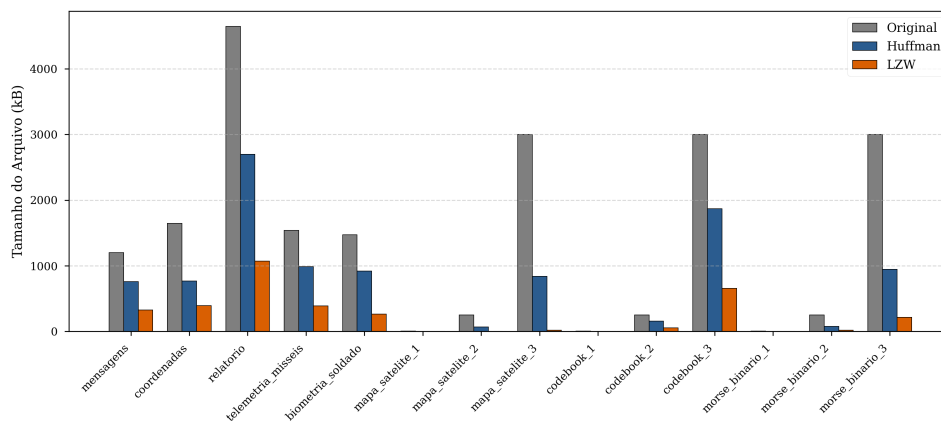


Figura 2. Redução do volume absoluto de dados (kB) por arquivo compactado.

é esperado, visto que o LZW realiza buscas e atualizações na tabela *hash* a cada novo símbolo processado, enquanto o Huffman, após a construção da árvore, apenas percorre uma estrutura já fixa.

De modo geral, os resultados evidenciam um equilíbrio (*trade-off*) entre duas prioridades distintas, que não têm o mesmo peso no contexto tático considerado. O LZW produz o arquivo comprimido final consideravelmente menor, o que reduz diretamente o tempo de antena ativa (*airtime*) e o consumo de banda necessários para transmitir a mensagem via rádio e, portanto, a janela de exposição à detecção pelo inimigo, ainda que ao custo de um tempo de processamento local maior antes da transmissão. O Huffman, por sua vez, comprime a mensagem mais rapidamente, exigindo menos tempo de CPU, o que sugere um consumo de energia proporcionalmente menor, mas produz um arquivo final maior, que exige mais tempo de antena ativa para ser transmitido. Como discutido na Explicação do Problema de Pesquisa, cenários táticos tipicamente priorizam a baixa probabilidade de interceptação e detecção (LPI/LPD) sobre a economia de processamento local, já que o custo de ser detectado tende a ser mais grave do que o custo adicional de bateria. Sob essa ótica, o LZW se mostra a escolha mais adequada para a maioria dos cenários avaliados neste estudo, restando o Huffman como alternativa preferível apenas em casos de hardware ou bateria criticamente limitados, nos quais o tempo de processamento

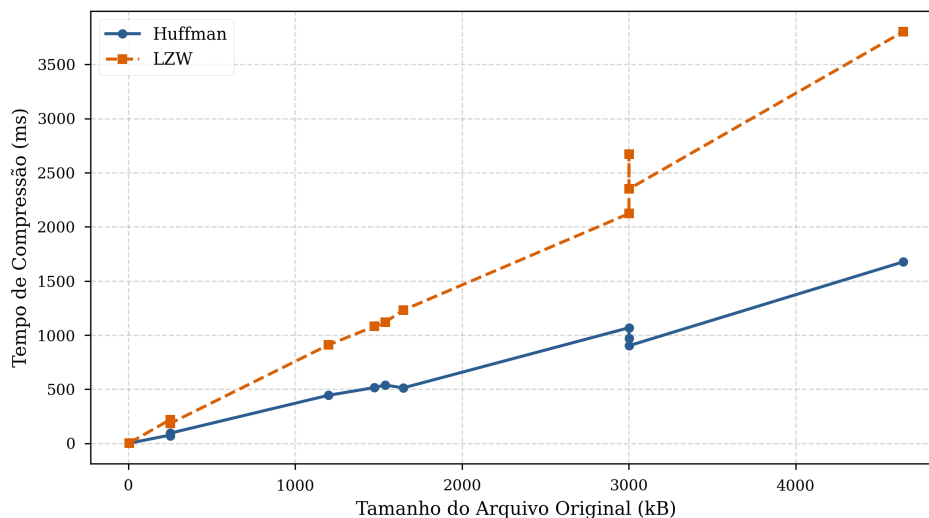


Figura 3. Tempo de execução da compressão (ms) em função do tamanho do arquivo original (kB).

adicional do LZW se torna proibitivo.

Em relação à facilidade de implementação, observou-se que o Huffman exigiu maior complexidade de codificação, uma vez que depende da construção explícita de uma árvore binária e de uma estrutura de fila de prioridades (*min-heap*), além de necessitar transmitir a árvore (ou a tabela de frequências) junto à mensagem para viabilizar a descompressão. Já o LZW mostrou-se mais direto de implementar, pois opera sobre uma única estrutura de tabela *hash* e não exige a transmissão de nenhuma informação além dos códigos, já que o dicionário é reconstruído de forma implícita e idêntica tanto na compressão quanto na descompressão. Esse menor acoplamento de estruturas de dados pode facilitar a manutenção do código em dispositivos de hardware limitado.

6. Conclusões Gerais e Trabalhos Futuros

Este artigo apresentou uma análise comparativa e empírica entre os algoritmos de compressão sem perdas LZW e Huffman, focando na aplicação em cenários de comunicação tática para a IoMT. O objetivo principal da pesquisa foi determinar a eficiência relativa de ambas as técnicas em termos de complexidade de tempo e espaço, taxa de compressão, tempos de execução prática e complexidade de implementação, utilizando conjuntos de dados sintéticos que simulam diferentes padrões de mensagens militares (como relatórios, coordenadas, mapas de satélite e transmissões em código Morse).

Os resultados experimentais demonstraram um *trade-off* entre as duas abordagens no contexto tático. O algoritmo LZW alcançou as maiores taxas de compressão absoluta, atingindo marcas superiores a 98% em arquivos com alta redundância interna, além de apresentar uma implementação mais direta e sem a necessidade de transmitir a estrutura de dicionário, características que reduzem o tempo de antena ativa. Em contrapartida, o algoritmo de Huffman destacou-se por sua velocidade, executando as etapas de compressão e descompressão substancialmente mais rápido, o que é vantajoso quando o hardware ou bateria são fatores limitantes. Como cenários táticos tipicamente priorizam a baixa probabilidade de interceptação e detecção sobre a economia de processamento

local, conclui-se que o LZW é a escolha mais adequada para a maioria dos cenários de comunicação tática avaliados, restando o Huffman como alternativa preferível apenas em casos de hardware ou bateria criticamente limitados.

Para trabalhos futuros, recomenda-se a validação das medições em hardware embarcado de baixa potência real e com restrição de processamento, tais como transceptores de rádio tático e nós IoT operacionais. Sugere-se também a avaliação do desempenho dos algoritmos sobre tráfego militar real e amostras de campo em vez de simulações sintéticas. Por fim, propõe-se a investigação de abordagens híbridas que combinem a velocidade do Huffman com a capacidade de redução volumétrica do LZW, bem como a análise do impacto direto do menor tempo de transmissão (*airtime*) na redução da probabilidade de interceptação de sinal por forças adversárias.

É possível acessar os nossos códigos por meio do link: https://github.com/Nichsz/Projeto_AED2_Comunicacao_tatica.

Referências

- CompuServe Incorporated (1990). Graphics interchange format, version 89a. <https://www.w3.org/Graphics/GIF/spec-gif89a.txt>.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. The MIT Press, Cambridge, MA, USA, 3rd edition.
- Cover, T. M. and Thomas, J. A. (2006). *Elements of Information Theory*. Wiley-Interscience, Hoboken, NJ, USA, 2nd edition.
- Dias, R. L., Boas, E. C. V., de Figueiredo, F. A. P., Mafra, S. B., and Ouameur, M. A. (2026). Data compression in lora networks: Performance and energy trade-offs of classical and cutting-edge compression algorithms. *Sensors*, 26(5):1414.
- Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101.
- Ketshabetswe, L. K., Zungeru, A. M., Mtengi, B., Lebekwe, C. K., and Prabakaran, S. (2021). Data compression algorithms for wireless sensor networks: A review and comparison. *IEEE Access*, 9:132715–132736.
- Krokosz, T., Rykowski, J., Zajęcka, M., Brzoza-Woch, R., and Rutkowski, L. (2023). Cryptographic algorithms with data shorter than the encryption key, based on lzw and huffman coding. *Sensors*, 23(17):7408.
- Nassra, I. and Capella, J. V. (2023). Data compression techniques in iot-enabled wireless body sensor networks: A systematic literature review and research trends for qos improvement. *Internet of Things*, 23:100806.
- Sedgewick, R. and Wayne, K. (2011). *Algorithms*. Addison-Wesley Professional, 4th edition.
- Welch, T. A. (1984). A technique for high-performance data compression. *IEEE Computer*, 17(6):8–19.
- Ziv, J. and Lempel, A. (1978). Compression of individual sequences via variable-rate coding. *IEEE Transactions on Information Theory*, 24(5):530–536.